

Lab02_Nakamura

Hannah Miranda Nakamura (249491)

Laboratório 02

```
library(dplyr)
```

Anexando pacote: 'dplyr'

Os seguintes objetos são mascarados por 'package:stats':

```
filter, lag
```

Os seguintes objetos são mascarados por 'package:base':

```
intersect, setdiff, setequal, union
```

```
library(readr)
```

Dos slides da aula temos que:

flights.csv.zip:

- Possui 5.714.008 observações;
- Tem 6 variáveis;
- Ocupa 1GB de RAM;
- Entre as variáveis:
 - Dia, mês, ano, dia da semana;

- Cia aérea, número do voo, registro do avião;
- Aeroportos de partida e de chegada;
- Horários de partida e chegada (reais e programados);
- Tempo de voo e distância voada;
- Atraso na chegada.

1) Quais são as estatísticas suficientes para a determinação do percentual de vôos atrasados na chegada (ARRIVAL_DELAY > 10)?

Dados as variáveis presentes no banco, é suficiente as estatísticas do número de atraso na chegada e o número total (n) de voos para poder calcular o percentual desejado (número de atrasos na chegada * 100 / número total de voos).

2) Crie uma função chamada getStats que, para um conjunto de qualquer tamanho de dados provenientes de flights.csv.zip, execute as seguintes tarefas (usando apenas verbos do dplyr:

```
getStats = function(input, pos){ # e) A função deve receber apenas dois argumentos:
  # i) input: o conjunto de dados (referente ao lote em questão);
  # ii) pos: argumento de posicionamento de ponteiro dentro da base de dados. Apesar de existir
  input |> filter(AIRLINE %in% c("AA", "DL", "UA", "US")) |> # a) Filtre o conjunto de dados
    filter(!is.na(YEAR),
           !is.na(MONTH),
           !is.na(DAY),
           !is.na(AIRLINE),
           !is.na(ARRIVAL_DELAY)) |> # b) Remova observações que tenham valores faltantes em
  group_by(YEAR, MONTH, DAY, AIRLINE) |> # c) Agrupe o conjunto de dados resultante de acordo com
  summarise(
    n_total_voos = n(),
    n_atrasos = sum(ARRIVAL_DELAY > 10),
    .groups = "drop"
  )# d) Para cada grupo em b., determine as estatísticas suficientes apontadas no item 1. c
}
```

3) Utilize alguma função readr::read_*_chunked para importar o arquivo flights.csv.zip.**

```

minhas_colunas <- cols_only(
  YEAR = col_integer(),
  MONTH = col_integer(),
  DAY = col_integer(),
  AIRLINE = col_character(),
  ARRIVAL_DELAY = col_double()
)

improtar_arquivo <- readr::read_csv_chunked(
  "flights.csv.zip",
  callback = DataFrameCallback$new(getStats), # b) Configure a função de callback para instanciar a função getStats
  chunk_size = 1e5, # a) Configure o tamanho do lote (chunk) para 100 mil registros;
  col_types = minhas_colunas # c) Configure o argumento col_types de forma que ele leia, diretamente, os tipos das colunas
)

```

4) Crie uma função chamada computeStats que:

```

computeStats = function(stats) {
  stats |>
    transmute( # b) Retorne as informações em um tibble contendo apenas as seguintes colunas
      Cia = AIRLINE, # i) Cia: sigla da companhia aérea;
      Data = as.Date(paste(YEAR, MONTH, DAY, sep = "-")), # ii) Data: data, no formato AAAA-MM-DD
      Perc = n_atrasos / n_total_voos,
      .group = "drop" # iii) Perc: percentual de atraso para aquela cia. aérea e data, apresentado em percentagem
    ) # a) Combine as estatísticas suficientes para compor a métrica final de interesse (percentual de atraso)
}

```

5) Produza um mapa de calor em formato de calendário para cada Cia. Aérea.

a) Instale e carregue os pacotes ggcal e ggplot2.

```

library(ggcal)
library(ggplot2)

```

b) Defina uma paleta de cores em modo gradiente. Utilize o comando `scale_fill_gradient`. A cor inicial da paleta deve ser `#4575b4` e a cor final, `#d73027`. A paleta deve ser armazenada no objeto `pal`.

```

pal <- scale_fill_gradient(low = "#4575b4", high = "#d73027")

```

c) Crie uma função chamada `baseCalendario` que recebe 2 argumentos a seguir: `stats` (tibble com resultados calculados na questão 4) e `cia` (sigla da Cia. Aérea de interesse). A função deverá:

```
baseCalendario <- function(stats, cia) {  
  dados_cia <- stats %>% filter(Cia == cia) # i) Criar um subconjunto de stats de forma a con  
  ggcal(dados_cia$Data, dados_cia$Perc) # ii) Para o subconjunto acima, montar a base do cal  
} # iii) Retornar para o usuário a base do calendário criada acima.
```

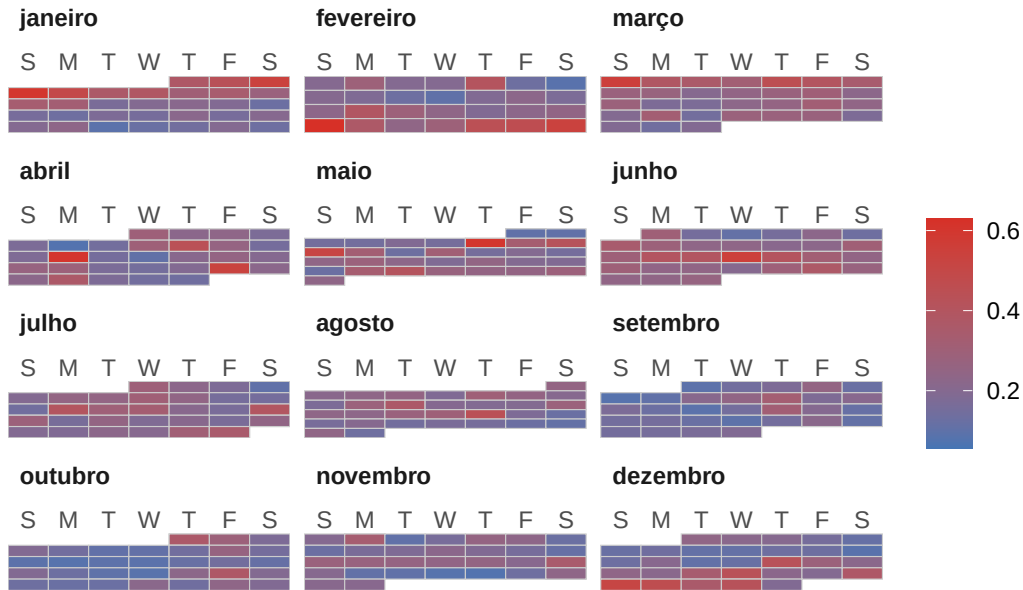
d) Executar a função `baseCalendario` para cada uma das Cias. Aéreas e armazenar os resultados, respectivamente, nas variáveis: `cAA`, `cDL`, `cUA` e `cUS`.

```
dados_finais <- computeStats(improtar_arquivo)  
  
cAA <- baseCalendario(dados_finais, "AA")  
cDL <- baseCalendario(dados_finais, "DL")  
cUA <- baseCalendario(dados_finais, "UA")  
cUS <- baseCalendario(dados_finais, "US")
```

e) Para cada uma das Cias. Aéreas, apresente o mapa de calor respectivo utilizando a combinação de camadas do `ggplot2`. Lembre-se de adicionar um título utilizando o comando `ggtitle`. Por exemplo, `cXX + pal + ggtitle("Titulo")`.

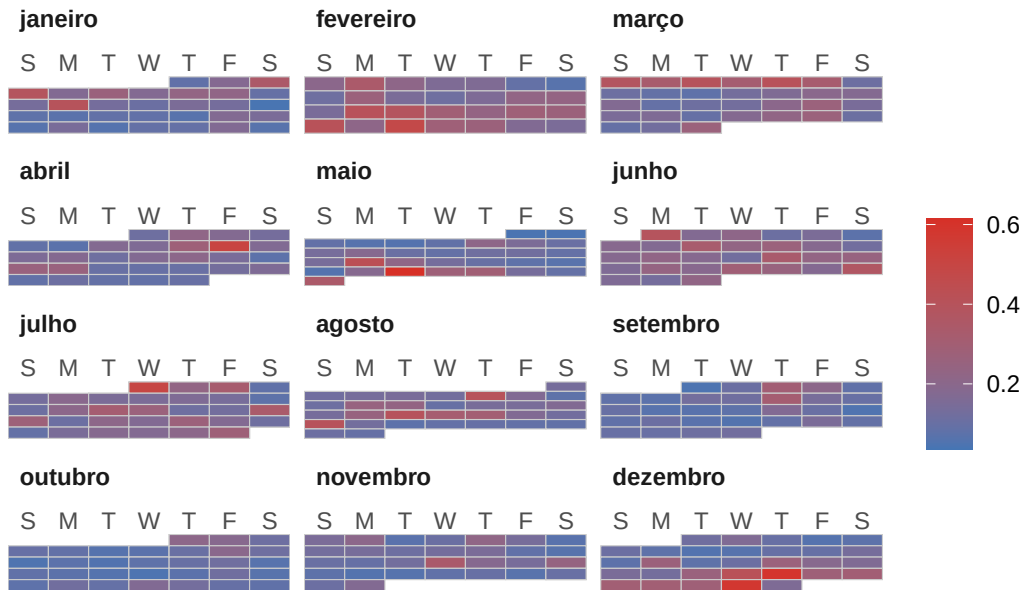
```
cAA + pal + ggtitle("Percentual de Atrasos na Chegada - American Airlines (AA)")
```

Percentual de Atrasos na Chegada – American Airlines (AA)



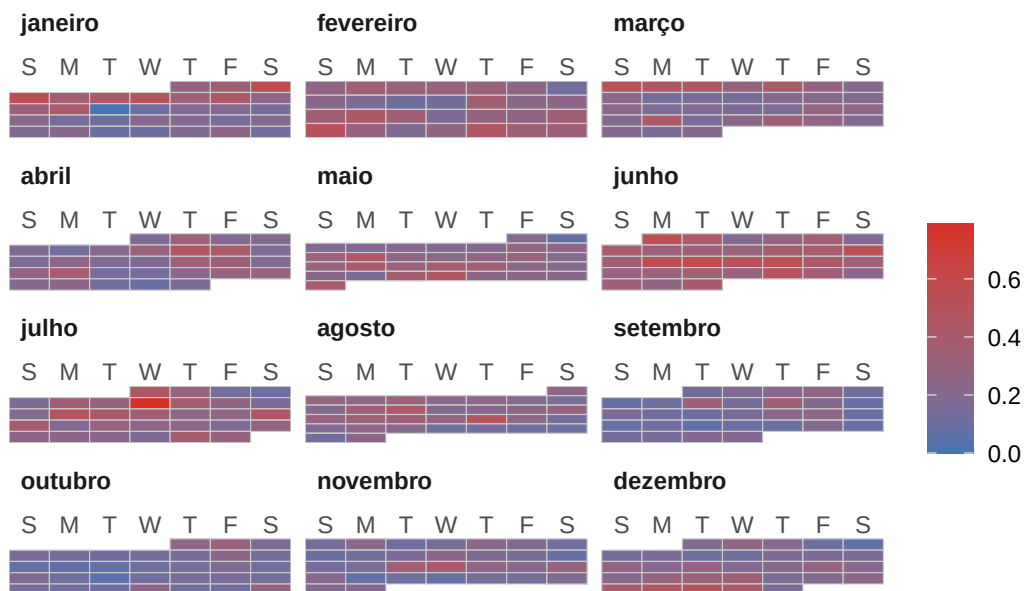
```
cDL + pal + ggtitle("Percentual de Atrasos na Chegada - Delta Air Lines (DL)")
```

Percentual de Atrasos na Chegada – Delta Air Lines (DL)



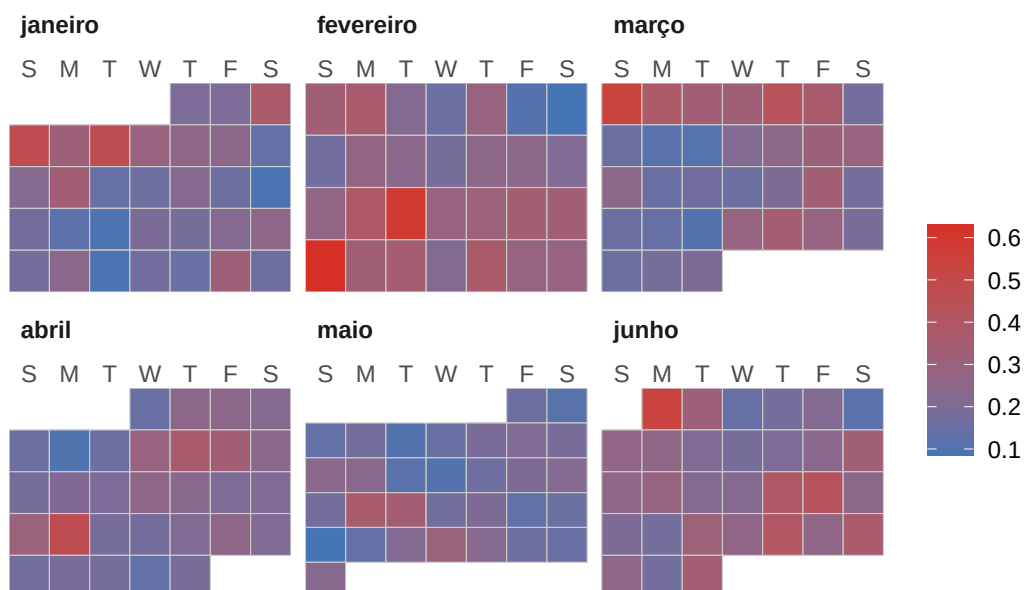
```
cUA + pal + ggtitle("Percentual de Atrasos na Chegada - United Airlines (UA)")
```

Percentual de Atrasos na Chegada – United Airlines (UA)



```
cUS + pal + ggtitle("Percentual de Atrasos na Chegada - US Airways (US)")
```

Percentual de Atrasos na Chegada – US Airways (US)



```
Sys.setenv(TZ = "America/Sao_Paulo")  
Sys.time()
```

```
[1] "2026-08-25 09:25:07 -03"
```